

ランダムテスト デバッグ 汎用テクニック

ランダムテストによるデバッグ

解説動画はこちらです。

1. 概要

競技プログラミングにおいて、バグを起こさずに素早く正確に実装するというのは非常に重要なスキルです。一方で、バグを全く起こさないというのは上級者にとっても現実的ではありません。したがって、バグを起こしてしまったときにそれを修正する**デバッグ**のスキルも非常に重要です。

デバッグが上手くできないと、ひとつの勘違いやミスの原因として膨大なコストを消費してしまいます。長時間考えても誤答の原因が分からず何も進展しないときは、精神的にも非常につらく感じられるでしょう。コンテストでの成績も、ミスが起こるかどうかに大きく左右される不安定なものになってしまいます。

残念ながら、そのような状況を常に解決できる万能な手法はありません。しかし、かなり多くの状況で**ランダムテスト**を行うことが有効で、競技プログラミングでも上級者のほとんどが日常的にこれを行っています。本講座では、この手法を簡単に解説します。

なお、本講座はデバッグ手法を解説するものですが、一方で、エディタやデバッガの使い方といったツールの解説はほとんど行いません。

2. 前提知識

必要な前提知識は特にありません。

3. サンプルの利用

問題の解答が実装できたら、まずは**サンプル**（入力例・出力例）を実行してみましょう。

この段階で正しく解けていないケースが見つかるのはある意味幸運です。サンプルは多くの場合、入力サイズが小さいため、手計算とプログラムの出力の照合が容易です。プログラムの中で計算された値を出力したり、計算の流れが想定通りかを確認していくと、実装ミスを特定しやすくなります。assert 関数などを用いることも効果的です。

この手順はほとんどの競技プログラミングユーザーが行っていることだと思います。

一方で、サンプルが合っているのに誤答だとジャッジされてしまうと、デバッグが困難になってしまうと感じる方も多いと思います。実際、「誤答になるケースを知りたい」「ジャッジのテストケースを知りたい」ということを考えたり聞いたりすることも珍しくないでしょう。

このようなときに有効なのが、次に述べるランダムテストです。

4. ランダムテスト

ランダムテストとは、サンプルケースの代わりに、乱数で自動生成したテストケースを使って、自分のプログラムの動作確認をする手法です。具体的には、次の 3 つを用意します。

- **プログラム A**：テストしたいプログラム。
- **プログラム B**：テストケースをランダム生成するプログラム。
- **プログラム C**：テストケースをプログラム A とは別の解法で解くプログラム。
 - このプログラムは、バグがなるべく起こりにくいもの、特にロジックが単純な全探索などの解法（**愚直解**）が通常用いられます。

この使い方は単純です。

- プログラム B によりテストケースを生成する。
- 生成したテストケースを、プログラム A、プログラム C で解く。
- これらのプログラムの出力が一致しなかった場合、生成したテストケースを出力する。

この手順によって出力が不一致となるようなテストケースが見つければ、これをサンプルとして利用してデバッグを行うことができます。

4.1. 注意点

- プログラム C の実装でバグが起こってしまうこともよくあります。しかしこの場合にも、（プログラム A と同一のバグらせ方をしてしまった場合を除き）見つけたテストケースを調べれば、少なくとも一方のプログラムのバグが発見できるため、あまり心配する必要はないと思います。
- プログラム C で実装する解法はユーザーのレベルによって様々です。少し複雑な全探索、簡単な dp や最短路問題が実装できると、効率よくランダムテストできる問題の幅がかなり広くなると思います。
- コンテスト中ではなく過去問に取り組んでいる状況であれば、プログラム C については他のユーザーの **AC している提出を利用する** という手もあります。はじめはこの方法を練習するのもよいと思います。

4.2. より具体的な手順

競技プログラミング上級者でも、方法は様々です。ここでは代表的な方法を 3 つ紹介します。

方法 1：3 つのプログラムを別ファイルに書く

例えば次の 3 つのプログラムを実装します。

- main.cpp：提出用のプログラム。
- generate.py：ランダムテストケースを生成するプログラム。
- naive.cpp：愚直解のプログラム。

さらに次のようなシェルスクリプトによって、main.cpp, naive.cpp が不一致になるケースを探します。

```
while true; do
  python3 generate.py > in.txt
  ./main < in.txt > out1.txt
  ./naive < in.txt > out2.txt
  if ! diff out1.txt out2.txt; then
    echo "found"
    break
```

```
fi
done
```

筆者が最もよく使っているのはこのタイプの方法です。提出用のプログラムに手を加えないでよいのがメリットだと思います。一方で、複数のファイルを編集しなければならないのが人によっては面倒に感じると思います。

方法 2：デバッグ時に標準入力からランダムテスト生成に切り替える

例えば C++ であれば、`#define` ディレクティブを利用した次のような実装が考えられます。

```
#define DEBUG

void solve(){
#ifdef DEBUG
    // テストケースをランダム生成する
#else
    // 標準入力からテストケースを入力する
#endif

    // 提出用の解法を実装する
    int ans1 = ...

#ifdef DEBUG
    // 愚直解を実装する
    int ans2 = ...
    if (ans1 != ans2){
        // output testcase, ans1, ans2
        exit(0);
    }
#endif
}
```

このような機能がない言語であれば、bool 変数を利用して同様のことが実現できるでしょう。

方法 3：機能ごとの関数を実装する

例えば次のようなプログラムが考えられます.

```
int solve(vector<int> A){
    // 提出用の解法を実装する
}

int naive(vector<int> A){
    // 愚直解を実装する
}

vector<int> generate_testcase(){
    // テストケースをランダム生成する
}

void test(){
    while(true){
        vector<int> A = generate_testcase();
        int ans1 = solve(A);
        int ans2 = naive(A);
        if(ans1 != ans2){
            // output testcase, ans1, ans2
            return;
        }
    }
}
```

1 ファイルで完結していて、ロジックが切り分けられているので読みやすいです. 一方で, 入力の受け取りと問題を解くコードが分離されている必要があるので, 普段のコーディングスタイルによっては少し手間が多く感じてしまうかもしれません.

4.3. 特殊なジャッジ形式の場合

ここまでの説明は, 入力に対して正答となる出力が一意に定まる問題を主に想定していました.

条件を満たす解を出力する形式の問題や, ジャッジとのやり取りを必要とするインタラクティブ問題の場合ではこれまでの方法がそのまま当てはめられないこともあるため, このような問題について簡単に説明します. これらのデバッグはどちらかというと, 上で解説してきた方法に慣れてきた人向けの手法となると思います.

とはいえ基本的な考え方はこれまでと同じで、

- ランダムな入力を作成し、
- プログラムが意図通りの動作をしているかを確認する

という手順がやはり有効です。このうち「意図通りの動作をしているかを確認する」という部分が、愚直解や AC を得ている提出との完全一致比較ではできないので、別の手段で置き換えればよいです。

条件を満たす解を出力する形式ならば、あなたの求めた出力が実際に条件を満たしていることを確認するための実装を追加すればよいです。解の存在判定やコスト最適化も必要ならば、その部分のみを愚直解との比較で行うこともできます。

インタラクティブ問題の場合には、ジャッジの対話と同等の機能をプログラム内に実装してしまうのが良いでしょう。次に、インタラクティブ問題の場合の実装例を挙げます。この実装は 4.2 節でいえば方法 2 にあたるものです。

- ABC299D の実装例：
<https://atcoder.jp/contests/abc299/submissions/71073587>

5. ランダムテストでバグが見つからない場合

バグの種類によっては、小さいランダムテストケースではバグが見つからないこともあります。例えば 100 万通りにランダム生成したケースすべてで正解を出力してしまう、というようなことも起こりえます。

このようなときにはデバッグの難易度が一気に跳ね上がってしまいます。基本的には、自分の解法の証明やひとつひとつの実装を見直すという地道な手順が必要になるでしょう。

ただし、ランダムテストでバグが見つからなかったという結果が得られたことは無意味ではありません。ここからどのようなバグの可能性が高いかについて考えることもある程度可能です。例えばサイズの小さいテストケースを生成しているのであれば、バグの原因としては、サイズが小さい場合特有のコーナーケースではないと考えるのが妥当です。

ランダムテストでバグが見つからなかった場合には、次のような点に特に気を付けてデバッグするとよいと思います。

- 問題文を誤読していないか確認しましょう。入出力の説明やサンプルの説明も含めて、全体を 1 文ずつ丁寧に読みなおしましょう。
- 生成していないテストケースがないかを確認しましょう。例えば以下のようなことが起こっていないかを確認しましょう。
 - $N = 1$ の場合を生成していない, A_i が 0 以下の場合を生成していない, 根付き木において親の頂点番号が子の頂点番号より大きいケースを生成していない, など。
- 愚直解での実装ミスをしていないかを確認しましょう。
- オーバーフローが起こっていないかどうかを確認しましょう。
- 配列外参照などの未定義動作が起こっていないかを確認しましょう。

また、問題によっては愚直解の計算量が大きすぎて、きわめて小さいサイズのランダムテストケースでしかテストできない場合もあります。このようなときには、より計算量のよい愚直解に置き換えられないか検討することも有効です。

6. 関連問題

練習問題として適している問題は特にありません。問題を解いている中で誤答してしまったときに、その問題について本講座の手法が適用できないかを考えてみるようにしてください。

7. まとめ

本講座では、ランダムテストによるデバッグについて解説しました。解説動画でもランダムテストを実践しているので、このような手法を聞いたことがなかった人や、やり方が分からず避けていた人も、ぜひ解説動画や本講座の内容を参考にして挑戦してみてください。

はじめはそれでもなかなかスムーズにデバッグできないことが多いかもしれませんが、ランダムテストによるデバッグを習慣化することで、あらゆる工程が上達していくと思います。また、自分でランダムテストケースを作ることに慣れると、デバッグに限らず

- 問題を解くために実験をして予想を立てる。
- TLE が出たときに入力のが大きさが最大のときの実行時間を調べる。

などが、心理的にも所要時間の意味でもスムーズに行えるようになるという副次的な効果も見込めます。

なお、目視でプログラムを確認するデバッグと、ランダムテストによるデバッグの優先度は、人によって様々です。ランダムテストは、目視でバグが見つからないときの最終手段と説明されていることも多いです。私はランダムテストを多用しており、目視でバグを探すよりも前にランダムテストを書くこともかなり多いです。いろいろなスタイルを試しながら、自分に合う方法を探してみてください。

8. 参考文献

1. seekworser. 1WAがとれない……ときのランダムテストのすゝめ.
<https://seekworser.hatenablog.com/entry/2022/10/04/001413>. (閲覧日: 2026-03-10).
2. e869120. デバッグ力を高める！ ～5 年間の経験から学んだ、競プロ・アルゴリズム実装におけるバグ取りの戦略～.
<https://qiita.com/e869120/items/8be6521e72025d7b2a13>. (閲覧日: 2026-03-10).
3. betrue12. 競プロでWAが出たときのランダム入力データ生成入門.
<https://betrue12.hateblo.jp/entry/2019/09/07/171628>. (閲覧日: 2026-03-10).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)